

DSA V2 — One Thing at a Time

18–24 Month JavaScript Problem-Solving Mastery Roadmap

Core philosophy: Finish one stage completely → pass the mastery gate → only then move to the next stage. Do not switch topics because a calendar says so.

1. What This Roadmap Is Designed To Build

This is not a “watch videos and finish 500 questions” plan. The target is independent problem solving. By the end, you should be able to receive an unfamiliar problem, identify what information matters, derive a brute-force idea, recognize a useful pattern or data structure, optimize it, prove or explain the approach, implement it in JavaScript, test edge cases, and state time/space complexity.

Estimated total: 18–24 months for serious mastery. A strong interview foundation can appear much earlier, around months 4–9, but mastery continues beyond that.

2. Your Learning Rule

- **Learn:** Use one primary teaching resource for the current topic.
- **Understand:** Explain the idea without looking at notes.
- **Implement:** Code the data structure/algorithm yourself.
- **Practice:** Easy → Medium → Hard where appropriate.
- **Blind test:** Solve unseen problems without being told the pattern.
- **Review:** Record mistakes and re-solve missed problems.
- **Mastery gate:** Do not advance until you can consistently start problems independently.

3. Where To Read / Learn

Resource	Use it for	How to use it
MDN Web Docs	JavaScript language, arrays, Map, Set, functions, recursion-related JS behavior	Primary JS reference. Read documentation when syntax/API behavior is unclear.
CodeWithHarry	JavaScript fundamentals and beginner-friendly DSA implementation	Primary learning/video resource for your JS-first path. Do not type along blindly.
Abdul Bari	Algorithms, intuition, correctness, complexity	Use selectively for difficult algorithmic concepts. Focus on why an algorithm works.
LeetCode	Interview-style problem solving	Main problem practice platform. Attempt first; use editorial/hints only after serious effort.
NeetCode	Pattern-based interview practice	Start seriously after your first core patterns are established; use it as a structured problem supplement, not another course to watch passively.
Striver A2Z	Comprehensive DSA syllabus and gap audit	Use later as a checklist/audit. Do not run it in parallel with every topic from Day 1.
Codeforces	Contest problem solving and speed	Introduce after fundamentals are strong. Use contests to train thinking under time pressure.

4. The Exact Study Loop For Every Topic

Use the same loop for Arrays, Hashing, Trees, DP, Graphs, and every other pattern.

1. Learn

20–40 minutes. Read/watch only enough to understand the concept. Pause frequently.

2. Close the resource

Explain the concept from memory. If you cannot, reopen only the missing part.

3. Implement

Write the basic operation/data structure yourself in JavaScript.

4. Brute force first

For each problem, identify the straightforward solution before trying to optimize.

5. Problem attempt

Spend roughly 20–30+ minutes before asking AI for help.

6. Escalate carefully

Own debugging → MDN/docs → search → one conceptual hint → approach review → solution only if necessary.

7. Re-code

If you saw a solution, close it and implement from memory. Then solve again later.

8. Complexity

Always state time and auxiliary space. Explain why.

9. Error journal

Record whether the failure was understanding, pattern recognition, logic, coding, complexity, or edge cases.

10. Blind test

After several guided problems, solve a problem where the pattern is not revealed.

5. Weekly Structure

Day	Focus
Monday	Learn the current concept + tiny implementation
Tuesday	Deep understanding + complexity + re-implementation
Wednesday	Basic/Easy problems
Thursday	Easy + first Medium; pattern recognition
Friday	Medium + review mistakes
Saturday	Deep practice: Medium/Hard + re-solves + implementation from blank editor
Sunday	Contest or timed set + 2–3 mixed problems + one blind problem + weekly review

Important: If a topic is not mastered by Sunday, the following week continues that same topic. The roadmap has no forced calendar deadline.

6. Full One-Thing-at-a-Time Roadmap

PHASE 0 — JavaScript Foundation — 3–4 weeks

- Variables, types, operators, conditionals, loops, functions, scope
- Arrays, strings, objects, destructuring, spread/rest
- Map, Set, Array methods, String methods, Object methods
- Sorting and comparison functions; basic recursion
- Coding-platform input/output conventions

MASTERY GATE: You can implement small JS programs from a blank file without following a tutorial line by line.

PHASE 1 — Logic Building — 4–6 weeks

- Numbers and digit problems
- Prime numbers, factors, GCD/LCM
- Palindromes, Armstrong numbers, Fibonacci
- Nested loops and pattern printing
- Arrays, strings, matrices
- Basic recursive thinking

MASTERY GATE: Target roughly 50–80 small problems. Given a basic problem, you can derive and test a solution without copying.

PHASE 2 — Time & Space Complexity — 1–2 weeks

- $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$
- Exponential and factorial growth
- Nested/sequential loops
- Recursion trees and call stack
- Input vs auxiliary space
- Complexity of common JS operations and algorithms

MASTERY GATE: Analyze about 20 pieces of code and explain both time and auxiliary space.

PHASE 3 — Arrays — 3–4 weeks

- Traversal, searching, insert/delete, reverse
- Frequency, duplicates, rotations
- Sorting basics
- Easy: Two Sum, Contains Duplicate, Stock, Maximum Subarray, Move Zeroes
- Medium: Product Except Self, 3Sum, Maximum Product, Subarray Sum K, Longest Consecutive
- Hard: First Missing Positive, Trapping Rain Water

MASTERY GATE: You can start an unseen array problem by identifying what state/information must be maintained.

PHASE 4 — Hashing — 2–3 weeks

- Map/Set and frequency counting
- Fast lookup and complement lookup
- Easy: Valid Anagram, Majority Element, Intersection
- Medium: Group Anagrams, Top K Frequent, Longest Consecutive, Subarray Sum K

MASTERY GATE: You naturally consider hashing when the problem needs fast lookup, frequency, membership, or complement information.

PHASE 5 — Two Pointers — 2 weeks

- Opposite-direction pointers
- Same-direction/fast-slow pointers
- Sorted-array elimination
- Valid Palindrome, Two Sum II, Reverse String, Squares
- 3Sum, Container With Most Water, Sort Colors, 4Sum
- Trapping Rain Water

MASTERY GATE: You can explain why two moving positions eliminate repeated work.

PHASE 6 — Sliding Window — 2–3 weeks

- Fixed window
- Variable window
- Frequency/state maintenance
- Expand/shrink invariant
- Longest Substring Without Repeating Characters
- Character Replacement, Permutation in String, Minimum Size Subarray Sum
- Fruit Into Baskets, Minimum Window Substring, Sliding Window Maximum

MASTERY GATE: You recognize contiguous-range problems where a maintained window avoids recomputation.

PHASE 7 — Prefix Sum — 1–2 weeks

- Running/prefix sum
- Range sum
- Prefix + HashMap
- Modulo prefix techniques
- Running Sum, Range Sum Query, Subarray Sum K, Continuous Subarray Sum, Subarray Divisible by K

MASTERY GATE: You can determine when cumulative state turns repeated range calculations into near-constant-time queries.

PHASE 8 — Binary Search — 3 weeks

- Classic binary search
- First/last occurrence
- Lower/upper-bound thinking
- Rotated arrays
- Binary search on answer/search space
- Koko Eating Bananas, Ship Packages
- Median of Two Sorted Arrays

MASTERY GATE: You can identify a monotonic search space and define correct low/high/invariant boundaries.

PHASE 9 — Linked List — 3 weeks

- Node/traversal
- Insert/delete
- Reverse
- Dummy nodes
- Fast/slow pointers
- Cycle detection
- Merge/reorder
- Reverse List, Merge Lists, Cycle, Middle, Remove Nth, Add Two Numbers
- Copy Random List, Sort List, Merge K Lists, Reverse K-Group

MASTERY GATE: You can manipulate pointers safely and explain every pointer update.

PHASE 10 — Stack — 2–3 weeks

- LIFO
- Stack implementation
- Monotonic stack
- Previous/next greater
- Expression evaluation
- Valid Parentheses, Min Stack, RPN, Daily Temperatures, Next Greater, Car Fleet
- Largest Rectangle in Histogram

MASTERY GATE: You can recognize nested/previous-next relationships and explain why a stack preserves useful candidates.

PHASE 11 — Queue/Deque — 1–2 weeks

- FIFO
- Circular queue
- Deque
- BFS foundation
- Monotonic deque
- Recent Calls, Queue Using Stacks, Circular Queue, Rotting Oranges, Sliding Window Maximum

MASTERY GATE: You can choose queue/deque when order of processing matters.

PHASE 12 — Recursion — 2–3 weeks

- Base/recursive case
- Call stack
- State and return values
- Recursion tree
- Recursive binary search
- Factorial, Fibonacci, sum, reverse, power, tree traversal

MASTERY GATE: You can trace recursion by hand and determine its base case, state, and complexity.

PHASE 13 — Backtracking — 3–4 weeks

- Choose → Explore → Undo
- Decision trees
- Subsets
- Letter Combinations
- Permutations
- Combination Sum I/II
- Generate Parentheses
- Word Search
- Palindrome Partitioning
- N-Queens
- Sudoku Solver

MASTERY GATE: You can define the state, choices, base case, and undo step without copying a template.

PHASE 14 — Trees — 4–5 weeks

- Binary tree representation
- DFS: preorder/inorder/postorder
- BFS/level order
- Height/diameter
- LCA
- Tree construction
- Serialization
- Maximum Depth, Same Tree, Invert, Symmetric, Diameter, Level Order
- Validate BST, Kth Smallest, Right Side View, Construct Tree, Serialize/Deserialize, Maximum Path Sum

MASTERY GATE: You can choose DFS/BFS and explain the state returned by recursive tree functions.

PHASE 15 — BST — 1–2 weeks

- BST invariant
- Search/insert/delete
- Inorder property
- Balanced vs unbalanced
- Validate BST, Kth Smallest, LCA, Insert, Delete

MASTERY GATE: You use the BST ordering property instead of treating it like an arbitrary binary tree.

PHASE 16 — Heap / Priority Queue — 2–3 weeks

- Min/max heap
- Heapify
- Priority queues
- Top K
- Streaming
- Kth Largest, Last Stone Weight, K Closest Points, Top K Frequent, Task Scheduler
- Median from Data Stream, Merge K Lists, IPO

MASTERY GATE: You know when maintaining only the best/worst K candidates is enough.

PHASE 17 — Intervals — 1–2 weeks

- Sort by start/end
- Overlap logic
- Merging
- Scheduling
- Meeting Rooms, Merge Intervals, Insert Interval, Non-overlapping Intervals, Meeting Rooms II, Arrows

MASTERY GATE: You can model overlap and ordering correctly before coding.

PHASE 18 — Greedy — 2–3 weeks

- Local vs global choice
- Sorting + greedy
- Exchange-argument intuition
- Assign Cookies, Jump Game I/II, Gas Station, Partition Labels, Task Scheduler, Candy

MASTERY GATE: You can explain why a local choice is safe rather than merely observing that it works on examples.

PHASE 19 — Graph BFS/DFS — 4–5 weeks

- Adjacency list/matrix
- DFS/BFS
- Visited state
- Connected components
- Cycle detection
- Grid graphs
- Path Exists, Flood Fill, Number of Islands, Clone Graph, Rotting Oranges, Course Schedule
- Pacific Atlantic, Surrounded Regions, Connected Components, Word Ladder

MASTERY GATE: You can translate a real problem into nodes/edges and choose traversal state correctly.

PHASE 20 — Topological Sort — 1–2 weeks

- DAG
- Indegree
- Kahn's algorithm
- DFS topological ordering
- Course Schedule, Course Schedule II, Alien Dictionary

MASTERY GATE: You recognize dependency ordering and can detect cycles.

PHASE 21 — Union Find / DSU — 1–2 weeks

- Parent arrays
- Find
- Union
- Path compression
- Union by rank/size
- Graph Valid Tree, Redundant Connection, Accounts Merge, Connected Components

MASTERY GATE: You can explain DSU as dynamic connectivity rather than memorizing code.

PHASE 22 — Trie — 1–2 weeks

- Trie nodes
- Insert/search
- Prefix search
- Word dictionary
- Implement Trie, Add/Search Words, Replace Words, Longest Word, Word Search II

MASTERY GATE: You understand why a trie is useful when prefixes are the central operation.

PHASE 23 — Bit Manipulation — 2 weeks

- AND/OR/XOR/NOT
- Shifts
- Set/clear/toggle bits
- Masks
- Counting bits
- Single Number, Number of 1 Bits, Counting Bits, Power of Two, Missing Number
- Sum of Two Integers, Bitwise AND, Subsets, Maximum XOR

MASTERY GATE: You can reason about binary representation rather than memorizing bit tricks.

PHASE 24 — Dynamic Programming — 8–12 weeks

- Recursion → memoization → tabulation → space optimization
- State definition and transition
- 1D: Climbing Stairs, House Robber, Decode Ways, Coin Change
- Grid: Unique Paths, Unique Paths II, Minimum Path Sum
- Knapsack: Partition Equal Subset Sum, Target Sum
- LIS, LCS, Edit Distance, Distinct Subsequences
- Palindromic subsequence, Burst Balloons, interval/tree/bitmask DP

MASTERY GATE: For a new DP problem, you can define state, transition, base cases, and iteration order yourself.

PHASE 25 — Advanced Graphs — 5–7 weeks

- Dijkstra
- Bellman-Ford
- Floyd-Warshall
- Kruskal
- Prim
- Strongly connected components
- Bridges
- Articulation points
- Eulerian path

MASTERY GATE: You can select an algorithm based on graph properties and edge constraints.

PHASE 26 — Advanced Data Structures — 5–7 weeks

- Segment Tree
- Fenwick Tree
- Sparse Table
- Monotonic Stack/Queue
- LRU Cache
- LFU Cache
- Range queries and update patterns

MASTERY GATE: You understand the operation/query trade-offs and can implement core structures from scratch.

FINAL PHASE — Random / Blind / Contest Mastery — 8–12+ weeks and ongoing

- Random LeetCode Easy/Medium/Hard
- Blind problems with no pattern label
- Striver A2Z completion/gap audit
- Codeforces contests
- Company problem sets
- Timed interview simulations
- Re-solve previously failed problems
- Explain solutions aloud

MASTERY GATE: A random unseen problem no longer requires someone to tell you the pattern. You can start the reasoning yourself.

7. When To Start NeetCode, Striver, LeetCode & Codeforces

- **LeetCode:** Start with the first serious DSA pattern (Arrays/Hashing). It is your main problem-practice platform.
- **NeetCode:** Start seriously after the first core patterns are established, around the Arrays → Hashing → Two Pointers → Sliding Window stage. Use it for pattern reinforcement and curated interview problems.
- **Striver A2Z:** Do not run it as a second full course from Day 1. Use it later as a comprehensive checklist/gap audit, then finish the missing sections systematically.
- **Codeforces:** Start regular contests after you have a basic foundation in arrays, hashing, two pointers, sliding window, binary search, recursion and common data structures. Increase contest frequency as your fundamentals improve.
- **Abdul Bari:** Use when you need deeper algorithmic intuition, proof, complexity, or a difficult concept—not for every single problem.
- **MDN:** Use whenever the blocker is JavaScript behavior/API knowledge rather than algorithmic reasoning.

8. AI Rules — Prevent AI Dependency

- Never paste a problem and immediately ask for the complete solution.
- Attempt for 20–30+ minutes first.
- Write the input/output and a manual example.
- Find a brute-force approach before optimizing.
- Identify the bottleneck yourself.
- Use console/debugger before asking AI to debug.
- If stuck, request only one conceptual hint.
- After seeing an approach, close it and re-code from memory.
- Re-solve missed problems after 1, 3, 7, 14 and 30 days.
- Keep some Sunday sessions completely AI-free.

9. Progress Tracking

Metric	What to record
Topic status	Not started / Learning / Practicing / Blind / Mastered
Problems	Attempted / solved independently / solved with hint / saw solution
Time	Minutes spent before help
Complexity	Correct time + auxiliary space explanation
Mistake type	Understanding / pattern / logic / coding / complexity / edge case
Re-solves	Day 1 / 3 / 7 / 14 / 30
Blind score	How many unseen problems you could start without pattern labels
Contest	Rating/result + mistakes to revisit

10. The 18–24 Month Mental Model

- Months 1–3:** JavaScript + logic + complexity + core array patterns.
- Months 4–6:** Binary search, linked lists, stacks, queues, recursion, backtracking, trees.
- Months 7–9:** BST, heaps, intervals, greedy, graphs, topological sort, DSU, trie, bits.
- Months 10–12:** Dynamic programming foundation and intermediate/advanced DP.
- Months 13–16:** Advanced graphs + advanced data structures + A2Z gap closing.
- Months 17–24:** Random problems, blind solving, contests, company sets, mock interviews and continuous refinement.

These are estimates, not deadlines. If a topic is not mastered, stay there. Consistency means continuing the process—not rushing to the next chapter.

11. Final Definition of DSA Mastery

You are not finished because you completed a playlist, a sheet, or a problem count. You are finished when you can repeatedly take unfamiliar problems and independently move through: **Understand** → **Example** → **Brute Force** → **Bottleneck** → **Pattern/Data Structure** → **Optimized Approach** → **Correctness** → **Code** → **Test** → **Complexity**.

One thing. Finish it. Prove it. Then move. ■